INSIDE STLC ACADEMY

PRACTICAL TEST AUTOMATION WITH PLAYWRIGHT

Fixtures & Hooks Guide

Set up and tear down cleanly with fixtures and hooks.

Introduction

Fixtures and hooks remove repetitive setup and keep tests focused. This guide explains when to use each so your suite stays DRY without becoming confusing.

Hooks

Hook	Runs
beforeEach	Before every test (common setup)
afterEach	After every test (cleanup)
beforeAll	Once before all tests in a file/describe
afterAll	Once after all tests

Custom Fixtures

Fixtures provide ready-made objects to tests — e.g. a logged-in page, or a page object. They are created on demand and torn down automatically, which is cleaner than repeated beforeEach logic.

Worked Example (logged-in fixture)

- export const test = base.extend({
- loggedInPage: async ({ page }, use) => {
- await login(page); await use(page);
- },
- });
- // test('...', async ({ loggedInPage }) => { /* already logged in */ })

Practical Exercise

Move repeated login steps from beforeEach into a custom fixture, and use it in two tests. Notice how much shorter the tests become.

Best Practices

- Use fixtures for reusable state (login, data, page objects).
- Keep setup close to where it is needed.
- Tear down what you create.

Common Mistakes

- Heavy beforeAll state shared (and mutated) across tests.
- Over-abstracting setup so tests are hard to follow.
- Forgetting teardown, leaking state.

> **INSIDE STLC PRO TIP**
>
> Fixtures beat beforeEach for reusable setup because they are composable and torn down automatically. Reach for a fixture as soon as two tests need the same starting state.